

From **How to Build a Course** — the full guide has the rest.

Appendix D — Building the Seven Features (Craft Notes)

*The seven-practices audit (§3) tells you **what** features your course needs; these tell you how to build a good version of each — one note per practice: why it's high-leverage, the moves, what AI can and can't do, and a worked example. Pull the one you need when a feature feels thin.*

Analogy Craft — teaching to explain simply

Analogy is how a hard idea becomes a simple one, and it may be the highest-leverage thing you do at the board — so it gets its own practice. A concept worth teaching earns **two** analogies, each doing a different job:

- **Analogy #1 builds the intuition** — it connects the new idea to something the audience already lives in.
- **Analogy #2 targets the misconception** — the specific place learners predictably go wrong — and bends the picture so the *wrong* turn feels wrong.

The skill underneath is **teaching to explain simply**, and it's worth practicing on purpose — and handing to students, because making *them* build the analogy is how you find out whether they actually understand the thing. AI can *generate* candidates (see §6, *Draft candidate analogies*), but a borrowed analogy in someone else's voice falls flat: **you** keep the one or two that ring true, rewrite them into your world and your audience's (a tech for tractors hears it differently than a data scientist does), and check that the mapping actually holds.

Worked example — backpropagation as seasoning a chicken. Training a model is like seasoning a chicken. You add a little salt and pepper, you taste, and the taste tells you *which way you were off* — too bland, too salty — so you adjust and taste again, closing in on right. That's backprop: **the seasoning is the weights** you're tuning, **the tasting is evaluating the loss** (how far off the dish is), and **the adjust-and-taste-again is the gradient step**, nudging each weight the direction the taste says to go. Round after round, the seasoning dials in. *Analogy #2, for the misconception:* people think the model lands on the answer in one shot. It doesn't — nobody seasons perfectly on the first shake. You *iterate*: taste, adjust, taste, adjust. The error isn't a failure; it's the information that tells you which way to move. Notice the move — warm, concrete, *theirs* (everyone has over-salted something), with a precise mapping underneath. That's the whole technique.

Structure Craft — turn a pile of facts into a map

Experts hold a subject as a *connected structure*; novices hold it as a *pile of facts* — and the pile is why they can't retrieve or transfer what they learned, relearning every case from scratch. Organizing the knowledge is what turns “I memorized twenty things” into “I see how this fits,” and it does double duty: the same map that helps students is what shows *you* the natural module boundaries (see §3.5). The moves:

- **Links are labeled *relationships*, not just lines.** A concept map is concepts joined by *named* relationships — “requires,” “is a kind of,” “causes,” “trades off against.” The label is where the learning lives: “*grind size* → *controls extraction* → *drives taste*” teaches more than the three terms sitting side by side.
- **Find the spine first.** Every domain has a backbone — a process, a cycle, a hierarchy, a cause-effect chain, a spectrum. Name the one organizing principle and hang everything off it. A map with no spine is just a tangle.
- **Show the whole before the parts.** The map isn't only your design tool — give students a version of it (a diagram, an advance organizer at the module intro, a “you are here”). When they see the structure first, each new piece has a place to land.
- **Test it by navigating.** A good structure lets a learner answer “*where does this fit?*” and “*what connects to what?*” If a concept has no links, or everything links to everything, the structure is wrong — fix it before you build content on top of it.

AI will happily *draw* a concept map — but it over-connects (everything relates to everything) and rarely picks the *one* spine that matters for your audience. Use it to surface candidate concepts and relationships; **you** choose the spine and prune to the links that carry weight.

Worked example — coffee, pile vs. map. The pile: *grind*, *water temperature*, *ratio*, *brew time*, *bean freshness* — five variables to memorize. The map: put **extraction** on the spine, and every variable becomes a labeled lever on it — “*grind finer* → *faster extraction* → *bitter if you overshoot*,” “*cooler water* → *slower extraction* → *sour if you undershoot*.” Now a beginner can *reason*: a sour cup means under-extraction, so grind finer or go hotter. The labeled cause-effect map *is* the understanding — and “extraction” is exactly the boundary that makes it one coherent module.

Motivation Craft — make them care before you teach

A learner who doesn't care won't learn — attention is the gate everything else passes through, which makes this the practice people most often *wave at* (“this matters because it's foundational”) instead of *building*. The research frame is **ARCS** (Keller): **A**ttention, **R**elevance, **C**onfidence, **S**atisfaction. In practice it's four moves:

- **Make relevance personal, not field-level.** “X is important in today’s economy” motivates no one. Connect to a stake *this* learner already has — a problem they hit, a goal they want, a frustration they feel. The test: can they finish “*I want this because ___*” in their own words?
- **Lead with the payoff, not the prerequisites.** People are pulled by the destination and repelled by the runway. Show the useful end result *first* — the working app, the café-quality cup — then say what it takes. Don’t make them earn the motivation by slogging through fundamentals.
- **Open on a real tension.** A hook works when learners *feel* the gap between where they are and where they want to be. “Everyone’s made a bad cup of coffee” surfaces a pain the course then resolves.
- **Engineer an early win.** Nothing motivates like “I just did that.” Design one real, reachable success in the first session (ARCS *Confidence*) — it buys you patience for the harder parts.

AI can draft candidate hooks, but relevance is **audience-specific** — one that lands for a data scientist falls flat for a tech maintaining tractors. You supply the audience truth; AI generates options; you keep the one that rings true for *your* people.

Worked example — a weak hook vs. a strong one. Weak: “*Machine learning is a foundational skill in today’s economy.*” (Field-relevance, no personal stake, no payoff, no tension — nobody leans in.) Strong: “*By the end of today you’ll have a model running that predicts house prices from real data — and you’ll see exactly where it’s wrong and why. No calculus. Now think of a prediction you’d actually want to make at work — we’ll build toward that.*” Notice the moves stacked: payoff first (“a model running today”), an early win (“by the end of today”), confidence (“no calculus”), and relevance made personal (“a prediction *you’d* want to make”).

Mastery Craft — show them what “good” looks like

People can’t hit a target they can’t see. Novices don’t know what *good* looks like, so they can’t aim for it or judge how far off they are — showing a complete, expert-quality example at the level to reach is one of the highest-return moves in teaching (the *worked-example effect*: studying a solved example beats grinding one out from scratch). The moves:

- **Show the whole, finished thing — the benchmark.** Not a fragment; the real deliverable, done well, so “good” is concrete. The Worked Examples that ship with this guide *are* this move — a full module at the bar to aim at.
- **Make the standard explicit.** Don’t just show it — say *why* it’s good (the two or three things that make it work), so learners can judge their own against the same criteria.
- **Show the process, not only the product.** Reveal how an expert got

there — the false starts, the checks, the “I’d redo this.” Otherwise learners think experts nail it first try, which is both false and demoralizing.

- **Then fade it.** They reproduce it with support, then without — that’s the scaffolding ramp (§5), and the benchmark is where the ramp starts.

AI drafts a plausibly-good example fast, but *good* is domain- and audience-specific; it tends to hand you the generically-competent version that misses what your field actually prizes. You set the bar and verify the example is genuinely exemplary — an off benchmark teaches the wrong target.

Worked example — the reference brew. The coffee course had no model of mastery until the audit forced one: an **instructor reference brew** students taste at the start (*this* is what you’re aiming at) and taste their own against. For a code course it’s the complete, verified pipeline notebook — the thing that runs, that they build toward.

Practice Craft — let them try before it counts

You don’t learn a skill by watching someone do it — you learn by doing it, getting feedback, and doing it again, *before* the stakes are real. It’s the step most courses skip: they explain, then assess, with no safe middle. The moves:

- **Practice the objective’s actual verb.** If the objective is to *implement* a pipeline, they practice implementing — not answering quiz questions *about* pipelines. Practice that doesn’t match the verb builds the wrong skill.
- **Keep it low-stakes.** Practice is for making mistakes cheaply; the moment it’s graded, it’s assessment, not practice. Protect a space where being wrong costs nothing.
- **Make feedback timely and specific.** “Good job” isn’t feedback; “*your split leaks test data into training — here’s why that inflates the score*” is. Feedback that arrives after it counts is too late to learn from.
- **Degrade it from the worked version.** The cleanest practice is the benchmark with holes cut in it (fill-in-the-blank, TODOs) — because it’s a working thing with parts removed, the practice can’t be subtly wrong.

AI generates practice items and even feedback well, but it pitches difficulty unevenly and defaults to generic notes; you set the difficulty ramp and aim the feedback at the specific misconception. (*Fuller build: the scaffolding ramp, §5, and build-complete-then-degrade, §6.*)

Worked example — the brew-along. A paired brew-along where the *tasting is the feedback* — immediate, specific, low-stakes — then a week-long brew log to practice alone before anything is assessed.

Climate Craft — make it safe to be a beginner

Fear shuts learning down. A learner afraid of looking stupid won't ask the question, try the risky thing, or admit they're lost — and every one of those is required to learn. Climate is the invisible variable that gates all the others, and it's where you grow the *person* into a practitioner, not just fill them with content. The moves:

- **Normalize struggle and error, out loud.** Say that confusion and mistakes are expected, and model your own — “debugging is normal,” “nobody seasons perfectly on the first shake.” A visible instructor misstep gives everyone permission.
- **Design for every learner, not the median.** Vary your examples (not all sports or war metaphors), don't assume background that quietly excludes, and offer more than one way in (this is UDL, §8). Ask: whose experience does each example assume?
- **Set the norms explicitly.** How questions work, how feedback is given (specific *and* kind), and the flat rule that *no question is too basic*. Say it in week one; hold to it.
- **Name the practitioner's traits and build reps for them.** Curiosity, skepticism, persistence — decide which a strong practitioner in your field has, then design moments that *exercise* them, not just mention them.

AI can scan your examples for assumed knowledge and suggest inclusive alternatives, but climate is set by *you* — in the room and in the tone of every artifact. It's a design *value* (see §0), not a feature bolted on at the end.

Worked example — open on a shared failure. The coffee course opens by having everyone recall a bad cup: now no one is the only novice, mistakes are the *subject*, and the room is safe to be wrong in. “Debugging is normal” does the same job for a code course.

Self-Direction Craft — set them up to keep going without you

A course ends; learning shouldn't. The real win is a learner who keeps improving after you're gone — anything less builds dependence, not capability, and that's what separates a course from a crutch. The moves:

- **Hand over your tools.** The cheat sheets, checklists, reference examples, and “how I'd debug this” heuristics you used — give them, so learners self-serve instead of coming back to you.
- **Teach the meta-skill, not just the answers.** How to find the answer, judge a source, run the next experiment — the fishing rod, not the fish. A learner who can *figure things out* outlasts one who memorized this year's facts.

- **Point to the next rung.** A curated “where to go deeper” — optional dives, a reading or two, the community — so momentum survives the last session instead of dying at the door.
- **Build in self-assessment.** A way for them to gauge their *own* understanding (the module’s self-check) so they can steer without you grading them.

The highest-leverage take-home now is teaching them to **keep learning with AI** — under the same guardrails you use in this course (it proposes; you verify). Model it, and they leave with a tutor that never closes.

Worked example — the take-home kit. Coffee: a recipe card + a “how to dial in *any* coffee” heuristic + a short resource list. Code: the cheat sheet + a “how to debug a model” checklist + pointers to the next topics — plus the habit of asking AI well.

Now go build one. Open `templates/Decisions_TEMPLATE.md` and fill the first page — audience, objective, depth. That single page is what everything in this guide hangs from, and it takes about twenty minutes. **Build one module end to end before you scale to the rest:** the first one is slow because you’re learning the method, and every one after it inherits the decisions you just made.